Algoritmos y estructuras de datos II

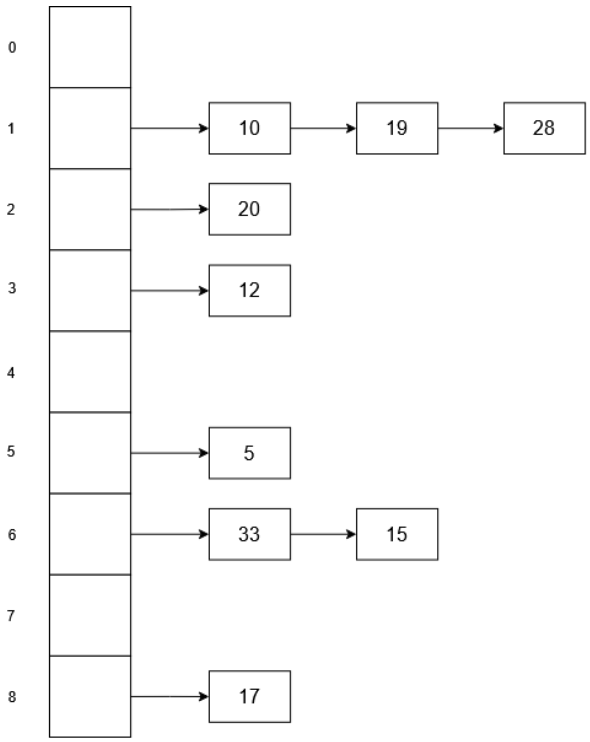
Trabajo Práctico: Hash Tables

PARTE 1

Ejercicio 1

Ejemplificar que pasa cuando insertamos las llaves 5, 28, 19, 15, 20, 33, 12, 17, 10 en un HashTable con la colisión resulta por el método de chaining. Permita que la tabla tenga 9 slots y la función de hash:

H (k) = k mod 9



PARTE 2

Ejercicio 3

Considerar una tabla hash de tamaño m = 1000 y una función de hash correspondiente al método de la multiplicación donde A = (sqrt(5)-1)/2). Calcular las ubicaciones para las claves 61, 62, 63, 64 y 65.

$$h(k) = \lfloor m(kA - \lfloor kA \rfloor) \rfloor$$

$$h(61) = \lfloor 1000\ (61 \cdot (\sqrt{5}-1)/2) - \lfloor 61 \cdot (\sqrt{5}-1)/2 \rfloor) \rfloor = \mathbf{700}$$

$$h(62) = \lfloor 1000\ (62 \cdot (\sqrt{5}-1)/2) - \lfloor 62 \cdot (\sqrt{5}-1)/2 \rfloor) \rfloor = \mathbf{318}$$

h(63) = ⌊1000 (63·(sqrt(5)-1)/2) - ⌊63·(sqrt(5)-1)/2)⌋)⌋ = **936**

h(64) = ⌊1000 (64·(sqrt(5)-1)/2) - ⌊64·(sqrt(5)-1)/2)⌋)⌋ = **554**

h(65) = ⌊1000 (65·(sqrt(5)-1)/2) - ⌊65·(sqrt(5)-1)/2)⌋)⌋ = **172**

Ejercicio 4

Implemente un algoritmo lo más eficiente posible que devuelva True o False a la siguiente proposición: dado dos strings $s_1 \ldots s_k$ y $p_1 \ldots p_k$, se quiere encontrar si los caracteres de $p_1 \ldots p_k$ corresponden a una permutación de $s_1 \ldots s_k$. Justificar el costo en tiempo de la solución propuesta.

Ejemplo 1:

Entrada: S = 'hola' , P = 'ahlo'

Salida: True, ya que P es una permutación de S

Ejemplo 2:

Entrada: S = 'hola' , P = 'ahdo'

Salida: Falso, ya que P tiene al carácter 'd' que no se encuentra en S por lo que no es una permutación de S

La implementación de este ejercicio se encuentra en tp-hashtable/code/ejercicios.py

La complejidad temporal de la solución propuesta es **O(n)**, siendo n la longitud de la cadena. Esto se debe a que recorremos una vez la cadena S insertando sus caracteres en la tabla, una vez la cadena P eliminándolos, y una vez los elementos de la tabla para comprobar que todos sean None. Esto resulta en O(n) + O(n) + O(n) = **O(n)**.

Ejercicio 5

Implemente un algoritmo que devuelva True si la lista que recibe de entrada tiene todos sus elementos únicos, y Falso en caso contrario. Justificar el costo en tiempo de la solución propuesta.

Ejemplo 1:

Entrada: L = [1,5,12,1,2]

Salida: Falso, L no tiene todos sus elementos únicos, el 1 se repite en la 1ra y 4ta posición

La implementación de este ejercicio se encuentra en tp-hashtable/code/ejercicios.py

Suponiendo que el valor del primo m produzca una distribución uniforme de claves al calcular h(k) = k mod m, la complejidad temporal resulta **O(n)**, siendo n el número de elementos de L. Una mala elección de m podría producir que todos los elementos terminen encadenados en la misma posición, produciendo una complejidad de **O(n^2)** en el peor caso. No obstante, como el primo m se elige a partir de la longitud de L, consideramos la complejidad temporal promedio como **O(n)**.

Ejercicio 6

Los nuevos códigos postales argentinos tienen la forma cddddccc, donde c indica un carácter (A - Z) y d indica un dígito 0, . . . , 9. Por ejemplo, C1024CWN es el código postal que representa a la calle XXXX a la altura 1024 en la Ciudad de Mendoza. Encontrar e implementar una función de hash apropiada para los códigos postales argentinos.

La implementación de este ejercicio se encuentra en tp-hashtable/code/ejercicios.py

Como el código postal siempre tiene exactamente **8 caracteres**, la función procesa una cantidad fija de datos, donde en cada iteración las operaciones aritméticas tienen tiempo constante, por lo que su complejidad temporal es **O(1)**.

Ejercicio 7

Implemente un algoritmo para realizar la compresión básica de cadenas utilizando el recuento de caracteres repetidos. Por ejemplo, la cadena 'aabcccccaaa' se convertiría en 'a2blc5a3'. Si la cadena "comprimida" no se vuelve más pequeña que la cadena original, su método debería devolver la cadena original. Puedes asumir que la cadena sólo tiene letras mayúsculas y minúsculas (a - z, A - Z). Justificar el costo en tiempo de la solución propuesta.

La implementación de este ejercicio se encuentra en tp-hashtable/code/ejercicios.py

En el primer bucle for se recorren los n elementos de la cadena s. Como las operaciones de search e insert tienen tiempo constante O(1), la complejidad de esta primera parte es O(n).

En el segundo bucle se recorre la tabla hash, que tiene 26 posiciones, y por lo tanto la complejidad temporal resulta O(n + 26) = **O(n)**.

Ejercicio 8

Se requiere encontrar la primera ocurrencia de un string $p_1 \dots p_k$ en uno más largo $s_1 \dots s_L$. Implementar esta estrategia de la forma más eficiente posible con un costo computacional menor a O(K*L) (solución por fuerza bruta). Justificar el coste en tiempo de la solución propuesta.

Ejemplo 1:

Entrada: S = 'abracadabra' , P = 'cada'

Salida: 4, índice de la primera ocurrencia de P dentro de S (abracadabra)

La implementación de este ejercicio se encuentra en tp-hashtable/code/ejercicios.py

La complejidad final de todo el algoritmo usando rolling hash es O(L+K), donde:

- L=|S|, la longitud del texto.
- K=|P|, la longitud del patrón.

Desglosemos:

Hash de P: O(K)

Hash de la primera ventana de S: O(K)

Desplazamiento de las ventanas:

Hay aproximadamente L−K desplazamientos y cada uno cuesta O(1): O(L−K)=O(L)

Por tanto: O(K) + O(K) + O(L−K) = O(L+K)

Y como K<L (exceptuando el caso donde la subcadena P es igual a la cadena S), podemos dar como complejidad final O(L).

Ejercicio 9

Considerar los conjuntos de enteros $S = \{s1, \dots, s_n\}$ y $T = \{t1, \dots, t_m\}$. Implemente un algoritmo que utilice una tabla de hash para determinar si S ⊆ T (S subconjunto de T). ¿Cuál es la complejidad temporal del caso promedio del algoritmo propuesto?

La implementación de este ejercicio se encuentra en tp-hashtable/code/ejercicios.py

Se realizan |T| inserciones para construir la tabla y hasta |S| búsquedas para comprobar los elementos de S. Si |T| >= |S|, la complejidad temporal promedio es dominada por la longitud de T, resultando en O(|T|). En el caso de que ambas listas tengan la misma longitud, la complejidad temporal asciende a O(|S| + |T|).

Ejercicio 10

Considerar la inserción de las siguientes llaves: 10; 22; 31; 4; 15; 28; 17; 88; 59 en una tabla hash de longitud m = 11 utilizando direccionamiento abierto con una función de hash h'(k) = k. Mostrar el resultado de insertar estas llaves utilizando:

1. **Linear probing**
2. **Quadratic probing con c1 = 1 y c2 = 3**
3. **Double hashing con h1(k) = k y h2(k) = 1 +(k mod (m - 1))**

1. Utilizando linear probing: $h(k,i) = (k + i) \bmod 11$

Elemento	Secuencia de posiciones	Posición final
10	10	10
22	0	0
31	9	9
4	4	4
15	4, 5	5
28	6	6
17	6, 7	7
88	0, 1	1
59	4, 5, 6, 7, 8	8

2. Utilizando quadratic probing: $h(k,i) = (k + i + 3 \cdot i^2) \bmod 11$

Elemento	Secuencia de posiciones	Posición final
10	10	10
22	0	0
31	9	9
4	4	4
15	4, 8	8
28	6	6
17	6, 10, 9, 3	3
88	0, 4, 3, 8, 8, 3, 4, 0, 2	2
59	4, 8, 7	7

3. Utilizando double hashing: $h(k,i) = (k + i \cdot h2(k)) \bmod 11$

Elemento	Secuencia de posiciones	Posición final
10	10	10
22	0	0
31	9	9
4	4	4
15	4, 10, 5	5
28	6	6
17	6, 3	3
88	0, 9, 7	7
59	4, 3, 2	2

Ejercicio 12

Las llaves 12, 18, 13, 2, 3, 23, 5 y 15 se insertan en una tabla hash inicialmente vacía de longitud 10 utilizando direccionamiento abierto con función hash h(k) = k mod 10 y exploración lineal (linear probing). ¿Cuál es la tabla hash resultante? Justifique.

La tabla resultante es la opción C:

0	
1	
2	12
3	13
4	2
5	3
6	23
7	5
8	18
9	15

Esta tabla hash es el resultado de aplicar linear probing de la forma

$$h(k) = (k + i) \bmod 10,$$

donde solo el último dígito importa. Se tiene la siguiente secuencia de posiciones:

Elemento	Secuencia de posiciones	Posición final
12	2	2
18	8	8
13	3	3
2	2, 3, 4	4
3	3, 4, 5	5
23	3, 4, 5, 6	6
5	5, 6, 7	7
15	5, 6, 7, 8, 9	9

Ejercicio 13

Una tabla hash de longitud 10 utiliza direccionamiento abierto con función hash h(k)=k mod 10, y exploración lineal (linear probing). Después de insertar 6 valores en una tabla hash vacía, la tabla es como se muestra a continuación.

0	
1	
2	42
3	23
4	34
5	52
6	46
7	33
8	
9	

¿Cuál de las siguientes opciones da un posible orden en el que las llaves podrían haber sido insertadas en la tabla? Justifique

(A) 46, 42, 34, 52, 23, 33

(B) 34, 42, 23, 52, 33, 46

(C) 46, 34, 42, 23, 52, 33

(D) 42, 46, 33, 23, 34, 52

La opción correcta es la C, puesto que se tiene la siguiente secuencia de posiciones:

Elemento	Secuencia de posiciones	Posición final
46	6	6
34	4	4
42	2	2
23	3	3
52	2, 3, 4, 5	5
33	3, 4, 5, 6, 7	7

Al igual que en el ejercicio 12, utilizar una función hash con módulo 10 provoca que solo el último dígito del número importe.